DESARROLLO DE INTERFACES

UT2: Creación, Empaquetado e Integración de Componentes Visuales (JavaBeans)

1. Concepto de Componente Visual y Graphics

Un **componente visual** es un módulo de código ya implementado, encapsulado y reutilizable que interactúa con otros elementos software a través de interfaces de comunicación. La metodología basada en estos elementos se denomina **Programación Orientada a Componentes (POC)**, permitiendo abaratar costes y reducir drásticamente los tiempos de desarrollo al emplear bloques previamente probados.

En Java, la clase abstracta `java.awt.Graphics` proporciona el contexto gráfico base para renderizar texto, geometrías primitivas e imágenes sobre lienzos visuales como `JPanel`, `Canvas` o `JFrame`.

2. Propiedades, Atributos y Ámbitos

Los **atributos** almacenan el estado interno y privado de una clase, mientras que las **propiedades** constituyen la interfaz pública observable y modificable mediante convenciones de nombrado `getPropiedad()` y `setPropiedad()` (o `isPropiedad()` para booleanos).

Criterio	Clasificación	Comportamiento en el Componente
Tipos de datos	**Propiedades Simples**	Almacenan un valor escalar atómico (ej. `int`, `String`, `boolean`).
	Propiedades Indexadas	Gestionan colecciones o vectores de datos (ej. las opciones de un `JComboBox`).
Ámbito	**Público (public)**	Accesible desde cualquier punto; define las propiedades expuestas.
	Privado (private)	Encapsulado; salvaguarda el estado interno del componente.
	Estático (static)	Compartido por todas las instancias de la clase sin instanciar el objeto.

3. Eventos y Escuchadores (Listeners)

Los eventos posibilitan la interactividad. Se dividen en la **detección del evento** y las **acciones de respuesta**. Todo evento requiere un *Listener* registrado que vigile el componente.

Evento	Listener Asociado	Métodos Clave y Desencadenante
`ActionEvent`	`ActionListener`	`actionPerformed`: pulsación de botones (`JButton`), tecla Enter en cajas (`JTextField`) o menú (`JMenuItem`).
`MouseEvent`	`MouseListener`	`mouseClicked`, `mouseEntered`, `mouseExited`, `mousePressed`, `mouseReleased`.
`MouseEvent`	`MouseMotionListener`	`mouseMoved` (movimiento sobre el control), `mouseDragged` (arrastre con clic pulsado).
`KeyEvent`	`KeyListener`	`keyPressed` (presionar tecla), `keyReleased` (soltar), `keyTyped` (carácter imprimible).
`FocusEvent`	`FocusListener`	`focusGained` (toma el foco activo), `focusLost` (pierde el foco hacia otro control).

4. Introspección, Reflexión y Persistencia

- **Introspección:** Capacidad de un entorno visual para consultar dinámicamente los metadatos de un componente (métodos, propiedades, eventos) mediante la clase `java.lang.Class` al arrastrarlo al lienzo.
- **Reflexión:** Permite actuar sobre dichos metadatos en tiempo de ejecución: instanciar clases dinámicamente (`newInstance()`), invocar métodos (`Method.invoke()`) y acceder a miembros privados (`setAccessible(true)`).
- **Persistencia y Serialización:** Permite "congelar" y almacenar en disco el estado configurado del componente en el diseñador gráfico sin necesidad de recompilar. Utiliza `java.io.Serializable` (automática) o `java.io.Externalizable` (programada).

5. Requisitos Obligatorios de un Componente Visual (JavaBean)

1. **Heredar de la jerarquía Swing:** Extender directamente de `JComponent`, `JPanel` (para componentes compuestos) o de un control estándar (`JTextField`, `JButton`).
2. **Constructor público sin parámetros:** El diseñador gráfico del IDE invoca internamente `new MiComponente()` al soltarlo en la vista de diseño. Si carece de este constructor, fallará la instanciación.
3. **Propiedades encapsuladas con Getters y Setters:** Todo atributo privado expuesto con `getPropiedad()` y `setPropiedad(...)` se integrará de forma automática en la ventana de propiedades del IDE.
4. **Implementar Serializable:** Obligatorio para la persistencia (las clases base de Swing como `Component` ya lo implementan por defecto).

6. Guía Práctica en Pasos: Creación e Integración en la Paleta

Paso 1: Definir la Clase y la Herencia Base

Crea una clase pública que extienda del control visual que deseas personalizar e implemente `Serializable`.

```
package miscomponentes;
import java.io.Serializable;
import javax.swing.JTextField;

public class JTextFieldNumerico extends JTextField implements Serializable {
    private int longitudMaxima = 10;
    // ...
}
```

Paso 2: Declarar el Constructor Público por Defecto

Establece los valores iniciales y los oyentes visuales (ej. cambio de fondo con `FocusListener` y filtrado de teclas con `KeyAdapter`).

```
    public JTextFieldNumerico() {
        super();
        this.addFocusListener(new java.awt.event.FocusAdapter() {
            @Override
            public void focusGained(java.awt.event.FocusEvent e) {
                setBackground(java.awt.Color.LIGHT_GRAY);
            }
            @Override
            public void focusLost(java.awt.event.FocusEvent e) {
                setBackground(java.awt.Color.WHITE);
            }
        });
    }
```

Paso 3: Exponer Propiedades mediante Métodos Get y Set

Cualquier propiedad que deba ser configurable desde el panel de propiedades del diseñador debe contar con sus accesores estándar.

```
    public int getLongitudMaxima() {
        return longitudMaxima;
    }

    public void setLongitudMaxima(int longitudMaxima) {
        this.longitudMaxima = longitudMaxima;
    }
```

Paso 4: Compilar y Generar el Paquete .JAR

Ejecuta la opción **Clean and Build** del IDE. El empaquetador creará el archivo ejecutable/biblioteca `.jar` dentro del directorio `dist/` (en proyectos estándar Ant) o `target/` (en proyectos Maven).

Paso 5: Importar el Componente a la Paleta de Controles

1. Accede al menú superior: `Tools → Palette → Swing/AWT Components`.
2. Pulsa el botón **Add from JAR...** y navega hasta la carpeta del archivo `.jar` generado.
3. Selecciona tu clase en la lista de componentes disponibles (ej. `JTextFieldNumerico`).
4. Elige la categoría de destino en la paleta (ej. *Swing Controls* o *Beans*) y confirma con **Finish**.

Paso 6: Uso Visual sobre Formularios JFrame

Abre cualquier formulario gráfico (`JFrame`). El nuevo componente aparecerá en la paleta lateral listo para ser arrastrado sobre el lienzo, permitiendo editar su `longitudMaxima` y apariencia directamente desde la ventana de propiedades sin escribir código manual.